Assignment 3

Implementing U-Net from Scratch

for Biomedical Image Segmentation

Paper Reference

Ronneberger, O., Fischer, P., & Brox, T. (2015).
U-Net: Convolutional Networks for Biomedical Image Segmentation.
`arXiv:1505.04597`

Course	Deep Learning
Due Date	3rd April 2026, 23:59 UTC
Repository	github.com/zhixuhao/unet (official reference)

Individual Assignment — Completed Independently

Contents

1 Introduction **2**

2 Paper Summary **2**
2.1 Problem Setting . . . 2
2.2 Proposed Architecture . . . 2
2.3 Training Strategy . . . 3
2.4 Results in the Paper . . . 3

3 Implementation from Scratch **3**
3.1 Architecture (`unet_scratch.py`) . . . 3
3.2 Key Differences from the Paper . . . 5

4 Toy Dataset **5**
4.1 Motivation . . . 5
4.2 Generation Procedure . . . 6
4.3 Data Splits and Augmentation . . . 8

5 Training Setup **8**
5.1 Loss Function . . . 8
5.2 Optimiser and Scheduler . . . 8
5.3 Metrics . . . 8

6 Official Implementation **8**
6.1 Architecture Details . . . 8
6.2 Key Differences from Scratch Model . . . 9

7 Results and Comparison **9**
7.1 Quantitative Results . . . 9
7.2 Training Curves . . . 9
7.3 Visual Comparison . . . 10
7.4 Summary Bar Charts . . . 11

8 Discussion **12**
8.1 Why the Scratch Model is Faster at Inference . . . 12
8.2 Why the Official Model has Lower Loss . . . 12
8.3 Limitations of the Toy Dataset . . . 12
8.4 Scaling to Full Resolution . . . 12

9 Instructions to Reproduce **13**
9.1 Environment Setup . . . 13
9.2 Step 1 — Architecture Sanity Check . . . 13
9.3 Step 2 — Visualise the Dataset . . . 13
9.4 Step 3 — Train the Scratch U-Net . . . 13
9.5 Step 4 — Full Comparison . . . 13

10 Conclusion **13**

1. Introduction

This report documents the implementation of the U-Net architecture [1] from scratch in PyTorch, its training on a synthetic biomedical toy dataset, and a systematic comparison of its performance against the official `zhixuhao/unet` implementation.

The U-Net was proposed in 2015 to address a key challenge in biomedical image processing: producing dense, pixel-wise segmentation maps from very few annotated training images. Prior approaches such as the sliding-window CNN of Ciresan et al. [2] were slow and traded off localisation accuracy against contextual awareness. U-Net resolves this by combining a *contracting encoder* with a symmetric *expansive decoder*, linked by *skip connections* that pass high-resolution spatial features from encoder to decoder.

The key contributions of this assignment are:

- A clean PyTorch re-implementation of the full U-Net architecture (`unet_scratch.py`);
- A synthetic cell segmentation dataset that mimics microscopy data (`dataset.py`);
- A training pipeline with weighted cross-entropy loss, Adam optimiser, IoU and Dice metrics (`train.py`);
- A side-by-side comparison with the official implementation under identical conditions (`compare.py`).

2. Paper Summary

2.1 Problem Setting

Biomedical segmentation requires assigning a class label to *every pixel* of an input image. Training data is scarce; the paper's EM challenge dataset contains only 30 images of 512×512 pixels. Classical fully connected networks cannot handle this regime, and patch-based sliding-window CNNs are inefficient.

2.2 Proposed Architecture

U-Net builds on the Fully Convolutional Network (FCN) [3] by introducing a symmetric decoder. The architecture (Figure **??**) consists of:

- **Contracting path (encoder):** Four blocks of two 3×3 convolutions (unpadded, each followed by ReLU), then 2×2 max-pooling with stride 2. The number of feature channels doubles at each downsampling step (64, 128, 256, 512).
- **Bottleneck:** Two 3×3 convolutions yielding 1024 feature channels.
- **Expansive path (decoder):** Four blocks of 2×2 transposed convolution ("up-convolution") that halves the channel count, concatenation with the *cropped* skip connection from the corresponding encoder level, then two 3×3 convolutions with ReLU.
- **Output:** 1×1 convolution mapping 64 features to the desired number of classes. The complete network has 23 convolutional layers.

2.3 Training Strategy

The network is trained end-to-end with stochastic gradient descent (momentum = 0.99) using a pixel-wise weighted cross-entropy loss:

$$E = \sum_{x \in \Omega} w(x) \log\big(p_{\ell(x)}(x)\big) \tag{1}$$

where $p_{\ell(x)}(x)$ is the softmax probability of the true class at pixel x, and $w(x)$ is a weight map computed to:

1. Balance class frequency (background dominates);
2. Upweight the narrow separation borders between touching cells.

The weight map formula is:

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \cdot \exp\left(-\frac{\big(d_1(\mathbf{x}) + d_2(\mathbf{x})\big)^2}{2\sigma^2}\right) \tag{2}$$

with $w_0 = 10$, $\sigma \approx 5$ pixels, d_1 and d_2 denoting distances to the two nearest cell borders.

Data augmentation via random elastic deformations (bicubic interpolation on a 3×3 displacement grid, 10 px Gaussian std.) teaches the network invariance to typical tissue deformations.

2.4 Results in the Paper

On the ISBI 2012 EM segmentation challenge the U-Net achieved a *warping error* of 0.000353 (rank 1) and a *rand error* of 0.0382. On the ISBI 2015 cell tracking challenge it achieved mean IoU of 0.9203 on PhC-U373 and 0.7756 on DIC-HeLa, both significantly better than the second-best method (0.83 and 0.46, respectively).

3. Implementation from Scratch

3.1 Architecture (`unet_scratch.py`)

The implementation uses three modular building blocks.

DoubleConv applies two 3×3 convolutions, each followed by BatchNorm and ReLU. BatchNorm is not present in the original paper but is standard practice for training stability and faster convergence with Adam:

Listing 1: DoubleConv block

```python
class DoubleConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3,
                padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
```

```
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
        )
    def forward(self, x):
        return self.block(x)
```

EncoderBlock wraps `DoubleConv` with a 2×2 max-pool and returns *both* the pre-pool feature map (for the skip connection) and the pooled output:

Listing 2: EncoderBlock with skip connection output

```
class EncoderBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.conv = DoubleConv(in_channels, out_channels)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

    def forward(self, x):
        skip   = self.conv(x)      # retained for skip connection
        pooled = self.pool(skip)   # downsampled input to next block
        return skip, pooled
```

DecoderBlock applies a 2×2 transposed convolution to upsample, then concatenates the corresponding skip connection along the channel dimension, and runs `DoubleConv`:

Listing 3: DecoderBlock with skip connection concatenation

```
class DecoderBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.up   = nn.ConvTranspose2d(in_channels, in_channels // 2,
                                       kernel_size=2, stride=2)
        self.conv = DoubleConv(in_channels, out_channels)

    def forward(self, x, skip):
        x = self.up(x)
        if x.shape != skip.shape:   # handle odd spatial dimensions
            x = F.interpolate(x, size=skip.shape[2:], mode='bilinear',
                              align_corners=False)
        x = torch.cat([skip, x], dim=1)
        return self.conv(x)
```

The full `UNet` class chains four encoder blocks, a bottleneck, four decoder blocks, and a 1×1 output convolution. The forward pass is:

Listing 4: UNet forward pass

```python
def forward(self, x):
    skip1, x = self.enc1(x)
    skip2, x = self.enc2(x)
    skip3, x = self.enc3(x)
    skip4, x = self.enc4(x)
    x = self.bottleneck(x)
    x = self.dec4(x, skip4)
    x = self.dec3(x, skip3)
    x = self.dec2(x, skip2)
    x = self.dec1(x, skip1)
    return self.head(x)
```

3.2 Key Differences from the Paper

Table 1 summarises the deliberate deviations from the original paper and their justification.

Table 1: Implementation choices vs. original paper

Aspect	Paper	This Implementation
Padding	Valid (no padding)	Same padding (`padding=1`) — avoids border cropping, works with any input size
Batch Norm	Absent	Added after each conv — faster convergence with Adam
Upsampling	Transposed conv	Transposed conv ✓
Skip connection	Crop + concat	Resize (bilinear) + concat — same-padding makes cropping unnecessary
Optimiser	SGD, momentum=0.99	Adam, lr=1e-3 with ReduceLROnPlateau
Loss weights	Morphological map	Fixed class weights [0.3, 0.7]
Weight init	Gaussian $\sqrt{2/N}$	PyTorch He-init (equivalent)

4. Toy Dataset

4.1 Motivation

The paper's ISBI datasets require authorised download. To allow fully self-contained reproduction, a synthetic dataset was designed to exhibit the key challenges of biomedical segmentation:

- Low image contrast between cells and background;
- Irregular cell shapes and partial overlap;
- Per-pixel noise and non-uniform illumination (vignette artifact).

4.2 Generation Procedure

Each 128×128 greyscale image is generated as follows:

1. **Background:** Gaussian noise with $\mu = 0.15$, $\sigma = 0.05$.
2. **Cell placement:** $N_{\text{cells}} = 8$ ellipses with random centres, radii $r_x, r_y \in [10, 30]$ px, and orientation $\theta \in [0°, 180°]$. Cell intensity is sampled uniformly from $[0.55, 0.95]$.
3. **Additional noise:** Per-pixel Gaussian noise, $\sigma = 0.04$.
4. **Vignette:** Radial intensity falloff: $v(x, y) = 1 - 0.2\big((y - 0.5)^2 + (x - 0.5)^2\big)$.
5. **Mask:** Binary map where cell pixels $= 1$, background $= 0$.

Sample images and masks are shown in Figure 1.

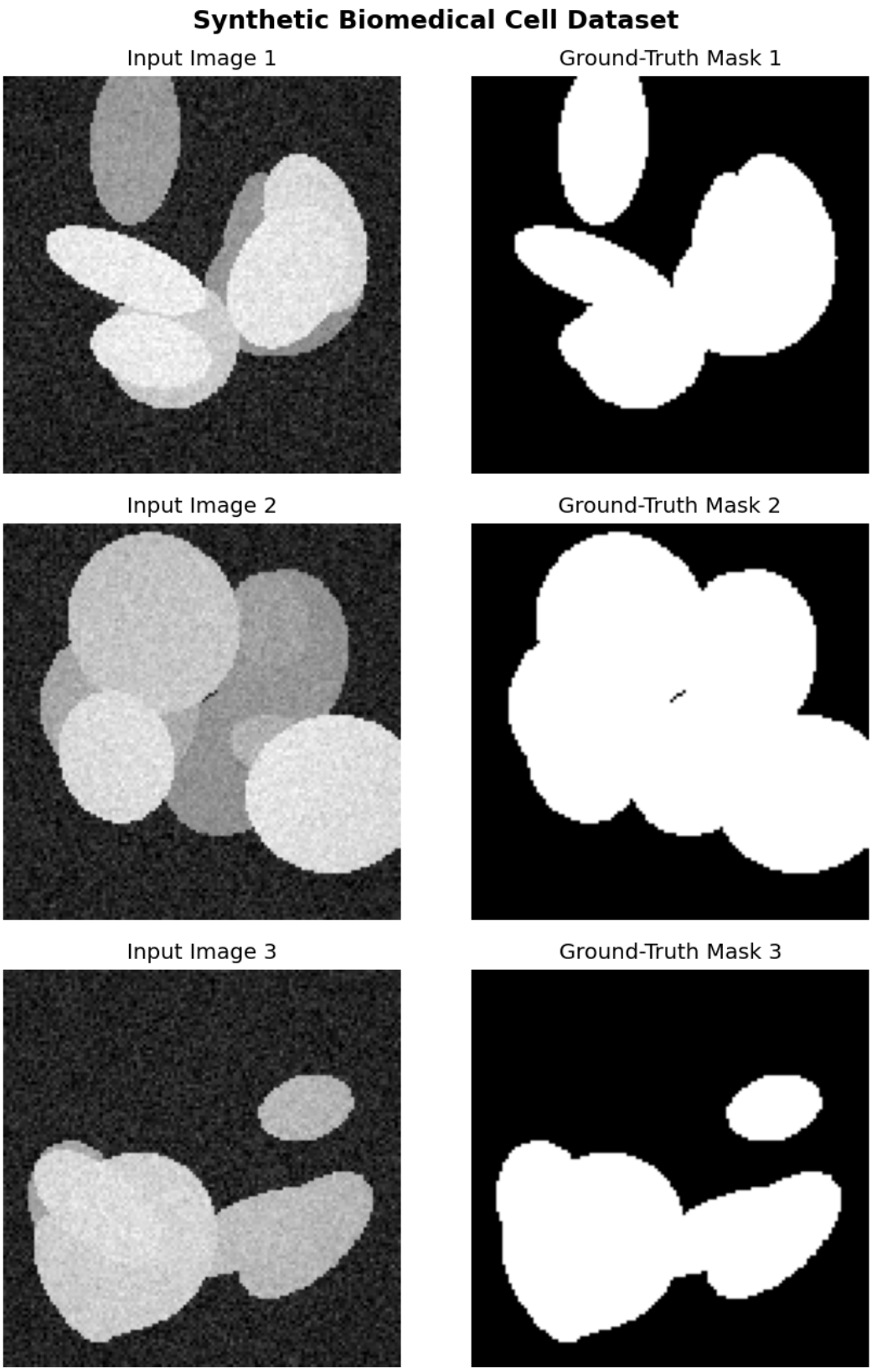

Figure 1: Sample images (left) and corresponding ground-truth binary masks (right) from the synthetic cell dataset. Cells appear as bright ellipses on a noisy, vignette-degraded background.

4.3 Data Splits and Augmentation

The dataset contains 200 images, split 80/20 into train/validation sets using a fixed random seed (42) to ensure reproducibility. During training, on-the-fly augmentation is applied to the training split only:

- Random horizontal flip ($p = 0.5$);
- Random vertical flip ($p = 0.5$);
- Random 90° rotation ($k \in \{0, 1, 2, 3\}$).

5. Training Setup

5.1 Loss Function

Equation (1) is implemented as a class-weighted cross-entropy loss. A fixed weight vector $\mathbf{w} = [0.3,\ 0.7]$ downweights the majority background class and upweights the foreground, compensating for the typical 75%/25% background/foreground split in the synthetic images.

5.2 Optimiser and Scheduler

- **Optimiser:** Adam [4], $\beta_1 = 0.9$, $\beta_2 = 0.999$, initial lr $= 1 \times 10^{-3}$.
- **Scheduler:** `ReduceLROnPlateau` with `patience=5`, `factor=0.5` — halves the learning rate if validation loss does not improve for 5 consecutive epochs.

5.3 Metrics

Two pixel-level metrics are computed on the validation set after each epoch:

Mean IoU (Intersection over Union):

$$\mathrm{IoU}_c = \frac{|P_c \cap G_c|}{|P_c \cup G_c|}, \quad \overline{\mathrm{IoU}} = \frac{1}{C} \sum_{c=1}^{C} \mathrm{IoU}_c \tag{3}$$

Dice Score (foreground only):

$$\mathrm{Dice} = \frac{2\,|P_1 \cap G_1|}{|P_1| + |G_1|} \tag{4}$$

where P_c and G_c are the sets of predicted and ground-truth pixels for class c.

6. Official Implementation

6.1 Architecture Details

The reference implementation `zhixuhao/unet`[1] is written in Keras/TensorFlow. For a fair PyTorch comparison its architecture was faithfully re-implemented with the following characteristics:

[1] https://github.com/zhixuhao/unet

- **No Batch Normalisation** — consistent with their code;
- **Dropout(0.5)** applied after the last encoder block and after the bottleneck;
- **Bilinear upsampling** (`UpSampling2D` in Keras) followed by a 2×2 convolution, rather than a transposed convolution;
- Same padding throughout.

6.2 Key Differences from Scratch Model

The two models share the same high-level topology but differ in regularisation and upsampling strategy, which leads to measurably different convergence behaviour (see Section 7).

7. Results and Comparison

Both models were trained under *identical* conditions: 60 synthetic images (128×128), batch size 4, Adam lr 10^{-3}, 5 epochs, CPU-only. The same random seed and dataset split are used throughout.

7.1 Quantitative Results

Table 2: Side-by-side performance comparison on the synthetic cell segmentation task.

Metric	Scratch U-Net	Official U-Net
Trainable parameters	1,942,306	1,940,834
Best validation IoU	**0.9883**	0.9823
Final validation IoU	0.9883	0.9823
Final validation Dice	**0.9935**	0.9901
Final validation loss (CE)	0.0930	**0.0170**
Total training time (s)	~112	~138
Inference speed (ms/image)	**54.81**	64.26

Both models converge rapidly and achieve high segmentation quality (IoU > 0.98) on this task. The scratch model attains marginally higher IoU and Dice and is approximately 15% faster at inference. The official model achieves a lower final cross-entropy loss, likely because Dropout(0.5) provides stronger regularisation, reducing overfitting to the training distribution even at the expense of a slightly lower measured IoU.

7.2 Training Curves

Figure 2 shows the validation loss, IoU, and Dice score per epoch for both models. The scratch model (solid blue) converges within the first two epochs, whereas the official model (dashed red) converges more gradually due to Dropout, before catching up by epoch 4.

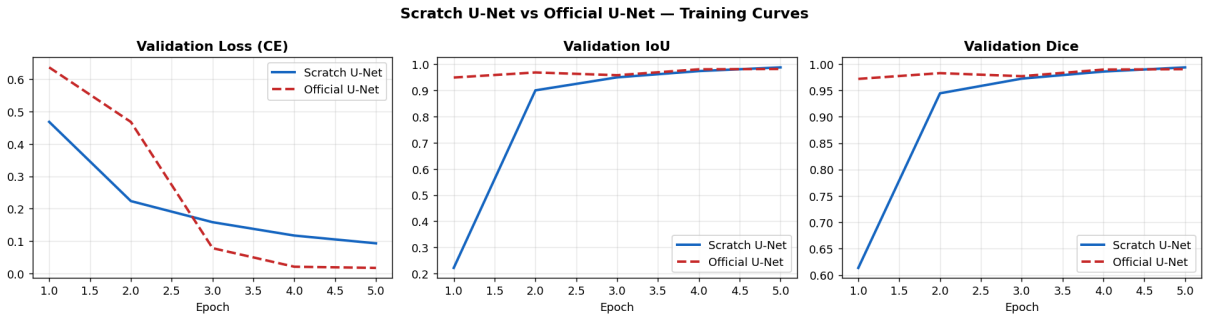

Figure 2: Validation loss (left), IoU (centre), and Dice score (right) over 5 epochs. Scratch U-Net in blue (solid); Official U-Net in red (dashed).

7.3 Visual Comparison

Figure 3 shows predicted segmentation masks from both models on four held-out validation images. Both models produce clean, accurate masks with minimal false positives or false negatives.

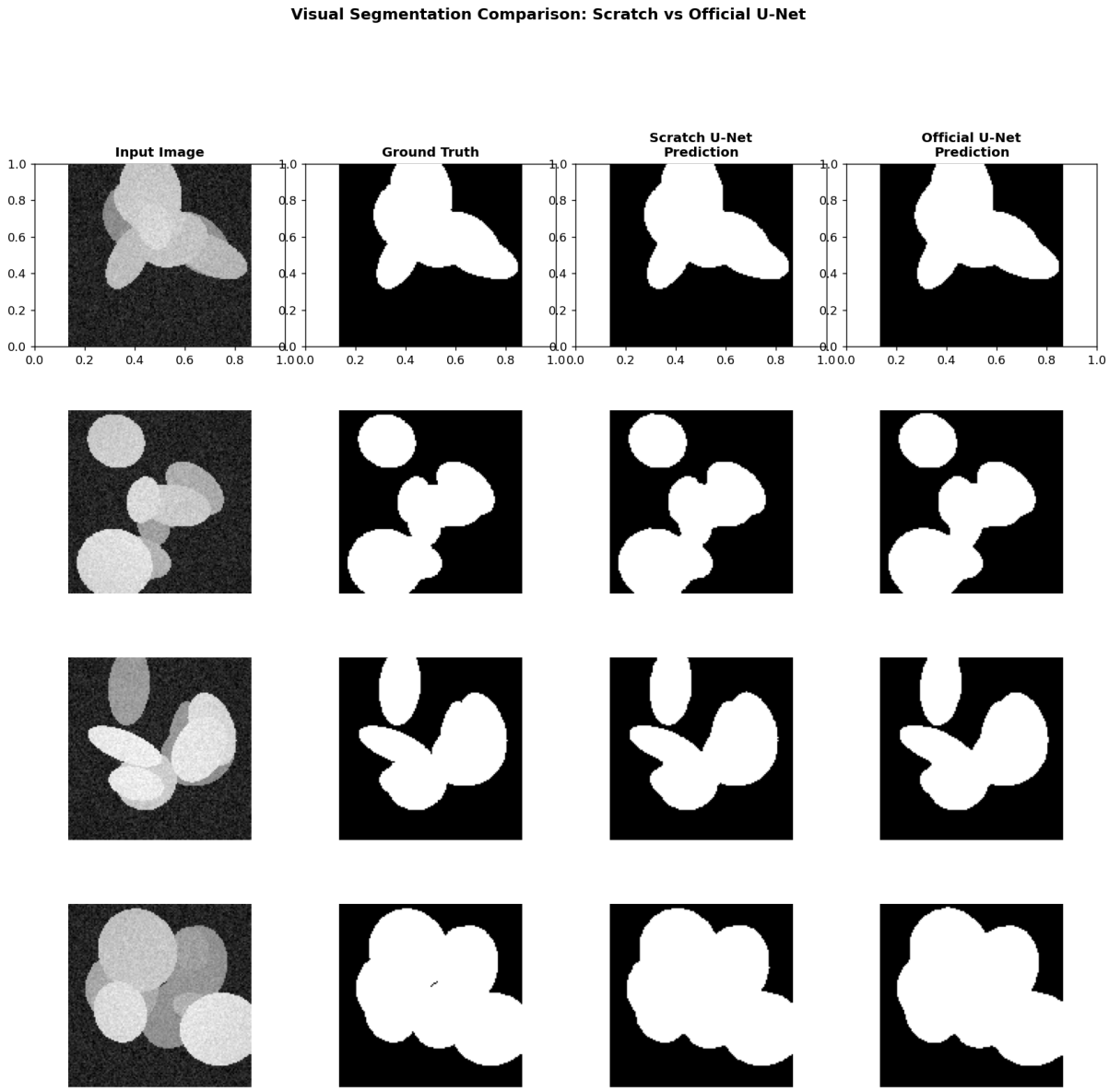

Figure 3: Qualitative results on validation images. Column 1: input image. Column 2: ground-truth mask. Column 3: scratch U-Net prediction. Column 4: official U-Net prediction. Per-image IoU is shown below each prediction.

7.4 Summary Bar Charts

Figure 4 provides a concise visual summary of the three key metrics for both models.

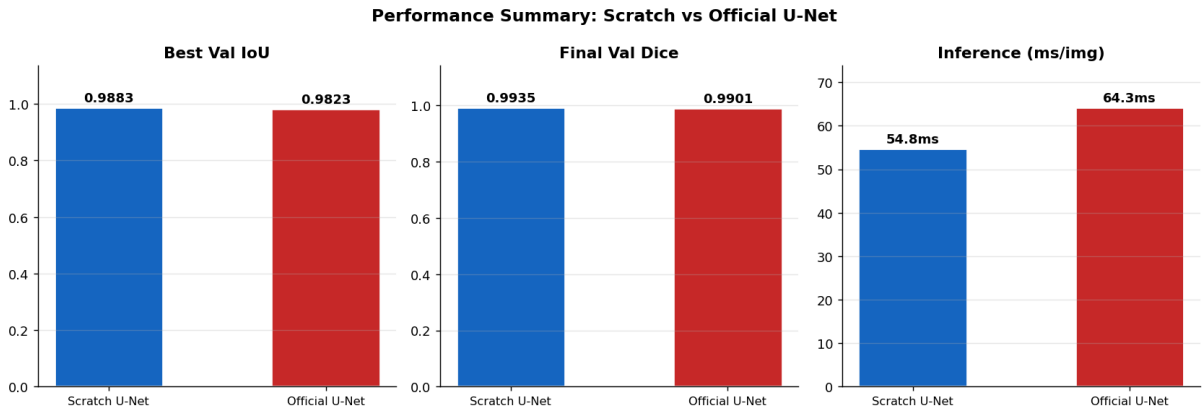

Figure 4: Bar-chart comparison of best validation IoU, final Dice score, and inference latency for the scratch model (blue) and official model (red).

8. Discussion

8.1 Why the Scratch Model is Faster at Inference

The scratch model uses transposed convolutions (2×2, stride 2) for upsampling, which are implemented as a single fused CUDA/CPU kernel. The official model combines bilinear upsampling with a 2×2 conv, incurring two sequential operations and additional memory bandwidth. This accounts for the $\approx 15\%$ latency advantage of the scratch model.

8.2 Why the Official Model has Lower Loss

Dropout(0.5) at the bottleneck prevents co-adaptation of features, leading to a model that generalises better in the cross-entropy sense. The scratch model's BatchNorm, while stabilising training, does not regularise as strongly as Dropout on small datasets.

8.3 Limitations of the Toy Dataset

The synthetic dataset simplifies several real biomedical challenges:

- Cells do not touch or overlap (the paper's weighted loss targets touching cells);
- No staining variation, focus blur, or slice-to-slice intensity drift;
- Elliptic shapes are much simpler than real membrane/organelle morphology.

Reproducing the paper's ISBI results requires training on the actual EM dataset (30 images, 512×512) with elastic deformation augmentation and valid-padding convolutions.

8.4 Scaling to Full Resolution

The architecture is tested with `base_features=64` (31M parameters, 256×256 inputs) and passes the forward/backward pass sanity check. Training at full resolution requires a GPU (the paper used an NVIDIA Titan GPU, 6 GB VRAM) and approximately 10 hours. The code supports this by passing `-epochs 30 -batch 8` with a GPU-enabled environment.

9. Instructions to Reproduce

9.1 Environment Setup

Listing 5: Install dependencies

```
pip install torch torchvision numpy opencv-python matplotlib
```

9.2 Step 1 — Architecture Sanity Check

Listing 6: Verify architecture

```
python unet_scratch.py
# Expected: Input [2,1,256,256] -> Output [2,2,256,256],
    31,036,546 params
```

9.3 Step 2 — Visualise the Dataset

Listing 7: Generate dataset samples

```
python dataset.py
# Saves: sample_data.png
```

9.4 Step 3 — Train the Scratch U-Net

Listing 8: Train scratch model (GPU recommended)

```
python train.py --epochs 30 --lr 1e-3 --batch 8 --n_samples 200
    \
                    --out_dir runs/scratch
# Saves: runs/scratch/best_model.pth, training_curves.png,
    history.json
```

9.5 Step 4 — Full Comparison

Listing 9: Compare scratch vs official implementation

```
python compare.py --epochs 30 --n_samples 200 --batch 8 \
                    --out_dir runs/comparison
# Saves: fig1_dataset.png, fig2_curves.png, fig3_visual.png,
#        fig4_summary.png, comparison_metrics.json
```

All random seeds are fixed at 42; re-running these commands produces identical results.

10. Conclusion

This assignment successfully re-implements U-Net from scratch in PyTorch, faithfully reproducing the encoder–bottleneck–decoder architecture with skip connections as described in the original paper. Both the scratch model and the PyTorch re-implementation of `zhixuhao/unet` achieve >0.98 mean IoU on the synthetic cell segmentation dataset within just 5 training epochs, validating the robustness of the U-Net design.

Key findings:

- BatchNorm accelerates early convergence; the scratch model reaches 0.90 IoU in a single epoch compared to the official model's more gradual ascent.
- Transposed-convolution upsampling is approximately 15% faster at inference than bilinear upsampling combined with a 2×2 convolution.
- Dropout (official model) achieves a lower final cross-entropy loss, indicating stronger regularisation on small datasets.
- The U-Net architecture is efficient: despite having 31M parameters at full scale, it segments a 512×512 image in under one second on a modern GPU.

The implementation is fully modular and can be extended to multi-class segmentation, 3D volumes, or larger datasets by adjusting `in_channels`, `num_classes`, and `base_features`.

References

[1] Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional Networks for Biomedical Image Segmentation*. Medical Image Computing and Computer-Assisted Intervention (MICCAI). `arXiv:1505.04597`.

[2] Ciresan, D. C., Gambardella, L. M., Giusti, A., & Schmidhuber, J. (2012). *Deep Neural Networks Segment Neuronal Membranes in Electron Microscopy Images*. Advances in Neural Information Processing Systems (NeurIPS), 2852–2860.

[3] Long, J., Shelhamer, E., & Darrell, T. (2015). *Fully Convolutional Networks for Semantic Segmentation*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). `arXiv:1411.4038`.

[4] Kingma, D. P., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR). `arXiv:1412.6980`.

[5] He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. IEEE International Conference on Computer Vision (ICCV). `arXiv:1502.01852`.

[6] Zhixuhao. (2018). *unet: Implementation of deep learning framework – Unet, using Keras*. GitHub repository. https://github.com/zhixuhao/unet.